Hackage: https://hackage.haskell.org/package/free
[Comonad reader] Free monads part 1: https://ekmett.github.io/reader/2011/free-monads-for-less/index.html
part 2: https://ekmett.github.io/reader/2011/free-monads-for-less-2/index.html
part 3: https://ekmett.github.io/reader/2011/free-monads-for-less-3/index.html

```
import Control.Monad.Free

-- MonadFree
class Monad m => MonadFree f m where
  wrap :: f (m a) -> m a
                      -- f (Free a) -> Free f a
liftF :: (Functor f, MonadFree f m) => f a -> m a
                      -- f a -> Free f a

-- Free
data Free f a where
  Pure :: a -> Free f a
  Free :: f (Free f a) -> Free f a

iter :: Functor f => (f a -> a) -> Free f a -> a
iterM :: (Monad m, Functor f) => (f (m a) -> m a) ->
Free f a -> m a           -- iterM f (Free fr) = f (iterM
f <$> fr)
foldFree :: Monad m => (forall x. f x -> m x) -> Free f
a -> m a              -- iterM (join . f)
retract :: Monad m => Free m a -> m a
                      -- iterM join

hoistFree :: Functor g => (forall x. f x -> g x) -> Free
f a -> Free g a
cutoff :: Functor f => Integer -> Free f a -> Free f
(Maybe a)
unfold :: Functor f => (b -> Either a (f b)) -> b ->
Free f a
unfoldM :: (Traversable f, Monad m) =>
  (b -> m (Either a (f b))) -> b -> m (Free f a)
```

Control.Monad.Free.Church

```
-- Control.Monad.Free.Church

data F f a where
 F :: { runF: forall r. ((a -> r) -> (f r -> r) -> r) }
-> F f a
instance Functor f => MonadFree f (F f)

improve :: Functor f => (forall m. MonadFree f m => m a)
-> Free f a
toF :: Free f a -> F f a
fromF :: MonadFree f m => F f a -> m a
```

```
iter :: (f a -> a) -> F f a -> a
                -- iter f (F p) = p id f
iterM :: Monad m => (f (m a) -> m a) -> F f a -> m a
                -- iterM f (F p) = p pure f
retract :: Monad m => F m a -> m a
                -- iterM join
foldF :: Monad m => (forall x. f x -> m x) -> F f a -> m
a               -- iterM (join . f)

hoistF :: (forall x. f x -> g x) -> F f x -> F g x
cutoff :: Int -> F f a -> F f (Maybe a)
```

Basics

```
-- Initialization
--
pure a :: Free f a                          -- a -> Free
f a
Pure a :: Free f a
wrap (y :: Free f a) :: Free f a            -- f (Free f
a) -> Free f a
Free (y :: Free f a) :: Free f a
-- derived
liftF (x :: f a) :: Free f a                -- f a ->
Free f a
wrap (pure <$> (x :: f a)) :: Free f a
```

==========

OLD

F
Free monad in **CPS style (aka codensity)**
i.e. fold function for Free

```
  data F f a where
    F :: forall r. ((a -> r) -> (f r -> r) -> r) -> F f a              (a -> r)       ~
Pure :: a -> Free f a
                                                                       (f r -> r)     ~
Free :: f (Free f a) -> Free f a

  -- e.g. if data A a = A a
  F (\kp _ -> kp 1) :: F A Int                                         ~ Pure 1  i.e. Free
equivalent                                  ~ 1
  F (\kp kf -> kf (A (kp 1)) :: F  A Int                               ~ Free (A (Pure 1))
                                            ~ A 1
  F (\kp kf -> kf (A (kf (A (kp 1 ))) :: F A Int                       ~ Free (A (Free (A
```

(Pure 1)))   ~ A (A 1)
  F (\kp kf -> kf (A (kf (A ... (kf (A (kf (A (kp 1))) ...)))   ~ Free (A (Free
(A ... (Free (A (Free (Pure 1))) ...)))   ~ A (A (A ... (A (A 1)) ...)))

F vs Free
F = **fold** of Free

```
  F :: forall r. ((a -> r) -> (f r -> r) -> r) -> F f a                F constructor
  foldFree :: (a -> r) -> (f r -> r) -> Free f a -> r                  Free fold
  F (\kp _ -> kp 1)                                                    ~ fold Pure 1
  foldFree kp _ (Pure 1) = kp 1
  F (\kp kf -> kf (A (kp1)))                                           ~ fold (Free (A (Pure
1))
  foldFree kp kf (Free (A (Pure 1))) = kf (A (kp 1)))


  -- F  <=>  Free conversion
  fromF :: F f a -> Free f a
  fromF (F f) = f Pure Free

  toF :: Free f a -> F f a
  toF (Pure a) = F $ \kp _ -> kp a
  toF (Free f) = F $ \kp kf -> kf (run kp kf . toF <$> f)
    where
      run kp kf (F x) = x kp kf
```

e.g. bind and fold with F

```
-- Setup
data A a = A a deriving Functor

liftA :: a -> F A a
liftA = liftF . A
```

Bind and fold with A

```
-- e.g. bind
F $ \kp _ -> kp 1                    -- pure 1 :: Free A
Int

F $ \kp kf -> kf $ A $               -- liftA 1 :: Free
A Int
               kp 1

F $ \kp kf -> kf (A $                -- liftA 1 >> liftA
2
               1 & (\_ ->
                kf $ A $
                  kp 2))

F $ \kp kf -> kf (A $                -- equivalent to
liftA 1 >> liftA 2
               kf $ A $
                kp 2)

F $ \kp kf -> kf (A $                -- liftA 1 >> liftA
```

```
2 >> liftA3
               1 & (\_ ->
                kf $ A $
                 2 & (\_ ->
                  kf $ A $
                   kp 3))
F $ \kp kf -> kf (A $                -- equivalent
               kf $ A $
                kf $ A $
                 kp 3)

kf A (kp 1)
kf A (1 & (\_ -> kf A kp 2))

-- fold
x :: F A Int
x = do
  liftA 1
  liftA 2
  liftA 3
runF x Pure Free = Free $ A $
                    Free $ A $
                     Free $ A $
                      Pure 3
```

Free object
A **free object** is a **decomposition** of an object to a **formal representation**:

```
  Monad f => (f a)
  data Free f a where                    free monad i.e. formal representation
of a monad
    Pure :: a -> Free f a                formal term x :: a
    Free :: f (Free f a) -> Free f a     formal monad f a

  data Pair a = MkPair a a
  data FreePair a where                                  free Pair i.e. formal
representation of Pair
    FPTerm :: a -> FreePair a                            formal term x :: a of
a pair
    FPPair :: (FreePair a) -> (FreePair a) -> FreePair a   formal MkPair x y ::
Pair a
```

A free object can easily add/remove **formal layer**:

```
  wrapF :: Pair (FreePair a) -> FreePair a               add FreePair layer
  wrapF (MkPair x y) = FPPair x y

  joinF :: FreePair (FreePair a) -> FreePair a           remove FreePair
layer
  joinF (FPTerm a) = a
  joinF (FPPair x y) = FPPair (joinF x) (joinF y)
```

e.g. Binding free monads x >> y

```
-- e.g Free with pair
```

```
-- f a = (Int, a)
Pure a
Free (0, Pure a)
Free (1, Free (2, Pure a))

x = Free (0, Free (1, Free (2, Pure "a")))
y = Free (3, Free (4, Pure "b"))
x >> y = Free (0, Free (1, Free (2, (Free (3, Free (4,
Pure "b"))))            append y to x


-- e.g. Free with function
-- f a = (Int -> a)
Pure a
x = Free (f >>> Pure)
y = Free (g >>> Pure)
x >> y = Free (f >>> const (Free (g >>> Pure))
                      append y to x
```

==========
Converting monad to free format

[Gonzalez] Why free monads matter: https://www.haskellforall.com/2012/06/you-could-have-invented-free-monads.html
[Parsons] What does free buy us: https://www.parsonsmatt.org/2017/09/22/what_does_free_buy_us.html
[Karpov] Free monads considered harmful: https://markkarpov.com/post/free-monad-considered-harmful.html
[Serokell] Intro to free monads: https://serokell.io/blog/introduction-to-free-monads#free-algebraic-structures
[Tweag] Free monads for interpreters: https://www.tweag.io/blog/2018-02-05-free-monads/

Conversion
Free monad **defers** the actual computation until fold.
Thus, a monadic function can be converted to free format by deferring the result to a **continuation** i.e. by using **CPS** style

```
doA :: m String                    simple monad
doB :: m ()                        no output
doC :: Int -> m String             input value
doD :: m Int -> m String           monadic input
doMap :: m x -> (x -> y) -> m y    mapping
doIO :: IO a -> m a                another monad e.g. IO

-- monad equivalent
data Do a where                                  a = dummy
  DoA :: (String-> a) -> Do a                    continuation with Int = Int -> a
  DoB :: (() -> a) -> Do a                       continuation with () = () -> a
  DoB' :: a -> Do a                              shorthand for CPS ()
  DoC :: Int -> (String -> a) -> Do a            continuation with String = (String -> a)
  DoD :: Int -> (String -> a) -> Do a
  DoMap :: x -> (x -> y) -> (y -> a) -> Do a     continuation with mapped y = (y -> a)
```

DoIO :: IO a -> Do a

e.g.

```
{-# LANGUAGE GADTs #-}

import Control.Monad.Free
import Control.Monad.IO.Class
import Data.Function
import Data.Functor.Classes

-- setup: original functions
doA :: MonadIO m => m String
doA = pure "A: Hello"

doB :: MonadIO m => m ()
doB = liftIO $ putStrLn "B: Hello"

doC :: MonadIO m => Int -> m Bool
doC = pure . even

doD :: (Show s, MonadIO m) => m s -> m String
doD = (show <$>)

doE :: Monad m => m a -> (a -> b) -> m b
doE = flip fmap

doIO :: MonadIO m => IO a -> m a
doIO = liftIO


-- setup: free data type for simulation
data Do a where
  DoA :: (String -> a) -> Do a
  DoB :: (() -> a) -> Do a
  DoB' :: a -> Do a
  DoC :: Int -> (Bool -> a) -> Do a
  DoD :: Show s => s -> (String -> a) -> Do a
  DoE :: x -> (x -> y) -> (y -> a) -> Do a
  DoIO :: IO a -> Do a

instance Functor Do where
  fmap f (DoA k) = DoA $ f . k
  fmap f (DoB k) = DoB $ f . k
  fmap f (DoB' a) = DoB' $ f a
  fmap f (DoC i k) = DoC i $ f . k
  fmap f (DoD s k) = DoD s $ f . k
  fmap f (DoE x g k) = DoE x g $ f . k
  fmap f (DoIO io) = DoIO $ f <$> io

instance Show1 Do where
  liftShowsPrec showA _ i (DoA f) = ("DoA " <>) . showA
i (f "")
  liftShowsPrec showA _ i (DoB f) = ("DoB " <>) . showA
i (f ())
```

```
  liftShowsPrec showA _ i (DoB' f) = ("DoB' " <>) .
showA i f
  liftShowsPrec showA _ i (DoC _ f) = ("DoC " <>) .
showA i (f True)
  liftShowsPrec showA _ i (DoD _ k) = ("DoD" <>) . showA
i (k "")
  liftShowsPrec showA _ i (DoE x f k) = ("DoE " <>) .
showA i (k $ f x)
  liftShowsPrec _ _ _ (DoIO _) = ("DoIO  " <>)


--- setup: Free
type DoF = Free Do


-- setup: simulating functions
doAF :: DoF String
doAF = liftF $ DoA id

doBF :: DoF ()
doBF = liftF $ DoB id

doB'F :: DoF ()
doB'F = liftF $ DoB' ()

doCF :: Int -> DoF Bool
doCF x = liftF $ DoC x id

doDF :: Show s => DoF s -> DoF String
doDF x = x >>= liftF . flip DoD id

doEF :: DoF x -> (x -> y) -> DoF y
doEF dx f = dx >>= \x -> liftF (DoE x f id)

doIOF :: IO a -> DoF a
doIOF = liftF . DoIO


-- fold
foldDoF :: forall m a. (MonadIO m) => DoF a -> m a
foldDoF = foldFree f
  where
    f :: Do x -> m x
    f (DoA k) = k <$> doA
    f (DoB k) = k <$> doB
    f (DoB' a) = doB >> return a
    f (DoC i k) = k <$> doC i
    f (DoD s k) = k <$> doD (pure s)
    f (DoE x g k) = k <$> doE (pure x) g
    f (DoIO io) = doIO io
```

Sample computation

```
(do
```

```
    x <- doAF
    doIO $ putStrLn x
    doBF
    doB'F
    doCF 1 >>= doIOF . putStrLn . ("1 even:" <>) . show
    doCF 2 >>= doIOF . putStrLn . ("2 even:" <>) . show
    doEF (pure (2 :: Int)) (* (2 :: Int)) >>= doIOF .
putStrLn . ("map:" <>) . show
    doDF (pure (1 :: Int)) >>= doIOF . putStrLn . ("pure
1:" <>)
  )
  & foldDoF


-- result
A: Hello
B: Hello
B: Hello
1 even:False
2 even:True
map: 4
pure 1:1
```

Example: IO simulation with Free
Use data
  data Action a where a = dummy variable to bind
Free
   PutStrLn :: String -> a -> Action a
   GetLine :: (String -> a) -> Action a

```
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE GADTs #-}

import Control.Monad.Free
import Control.Monad.IO.Class


-- step: define simulator
data Action a where
  PutStrLn :: String -> a -> Action a
  GetLine :: (String -> a) -> Action a
  deriving Functor


-- step: wrap in Free
type ActionF = Free Action
putStrLnF :: String -> ActionF ()
putStrLnF s = liftF $ PutStrLn s ()

getLineF :: ActionF String
getLineF = liftF $ GetLine id
```

```
-- step: compute with Free
c :: ActionF ()
c = do
  putStrLnF "Hello"
  putStrLnF "World"
  x <- getLineF
  y <- getLineF
  putStrLnF ("x=" <> x <> ", y=" y)
```

interpret ActionF

```
-- e.g. interpret Free action as IO
foldToIO :: ActionF a -> IO a
foldToIO = iterM h
  where
   h (GetLine k) = putStr "Enter value:" >> getLine >>=
k
   h (PutStrLn s a) = putStrLn s >> a


-- e.g. interpret Free action as String
foldToString :: forall a. ActionF a -> String
foldToString (Pure _) = ""
foldToString (Free (PutStrLn s a)) = s <> ":" <>
foldString a
foldToString (Free (GetLine next)) = "GetLine 100:" <>
foldString (next "100")
```

Output

```
 -- foldActionIO c
Hello
World
Enter value: 1
Enter value 2
x=1,y=2


-- foldToString c
Hello:World:GetLine 100:READ 100:x=100, y=100
```

e.g.

```
c :: ActionF ()
c = do
  putStrLnF "Hello"
  putStrLnF "World"
  x <- getLineF
  y <- getLineF
  printF ("x=" <> x <> ", y=" y)
```

```
-- expanded
Free $ PutStrLn "Hello" $
putStrLnF "Hello"
 Pure ()

Free $ PutStrLn "Hello" $
 putStrLnF "World
 Free $ PutStrLn "World" $
  Pure ()

Free $ PutStrLn "Hello" $                x <- getLineF
 Free $ PutStrLn "World" $
  Free $ GetLine $ id >>>
   Pure

Free $ PutStrLn "Hello" $                y <- getLineF
 Free $ PutStrLn "World" $
  Free $ GetLine $ id >>> (\x ->
   Free (GetLine $ id >>>
    Pure
   )
  )

Free $ PutStrLn "Hello" $               putStrLnF ("x="
<> x <> ", y=" y)
 Free $ PutStrLn "World" $
  Free $ GetLine $ id >>> (\x ->
   Free (GetLine $ id >>> (\y ->
    Free (PutStrLn ("x=" <> x <> ", y=" y) $
     Pure ()
    )
   )
  )
```

Example: Simulate state, reader, writer
Use

```
data Action w s a where                  a = dummy variable to bind Free
  -- simulate State
  Get :: (s -> a) -> Action w s a
  Put :: s -> a -> Action w s a

  -- simulate Writer
  Tell :: w -> a -> Action w s a
```

```
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE GADTs #-}

import Control.Monad.Free
import Control.Monad.IO.Class
import Control.Monad.RWS.Class
import Control.Monad.Trans.RWS (RWST, runRWST)
```

```
-- Step: define data
data Action w s a where
  Get :: (s -> a) -> Action w s a
  Put :: s -> a -> Action w s a
  Tell :: w -> a -> Action w s a
  deriving (Functor)


-- Step: Free constructor
type ActionF w s = Free (Action w s)

tellF :: w -> ActionF w s ()
tellF w = liftF $ Tell w ()

getF :: ActionF w s s
getF = liftF $ Get id

putF :: s -> ActionF w s ()
putF s = liftF $ Put s ()


-- Step:
foldRWS :: (MonadState s m, MonadWriter w m, MonadIO m)
=> ActionF w s a -> m a
foldRWS = iterM f
  where
    f (Get k) = fPutStr "Get:" >> get >>= k
    f (Put s a) = put s >> fPutStr "Put:" >> a
    f (Tell w a) = tell w >> a
    fPutStr msg = liftIO $ putStrLn msg
```

e.g.

```
c :: ActionF String Int ()
c = do
  tellF "INFO: Start"
  getF >>= (tellF . ("current state:" <>)) . show
  getF >>= putF . (+ 100)
  getF >>= (tellF . ("updated state:" <>)) . show
  tellF "INFO: end"
  pure ()

(((), lg), state) <- runState (runWriter $ foldRWS c) 0
   -- initial state = 0, final state = 100

   -- lg = "INFO: Start current state: 0 updated state:
100 INFO: end"
```